so [aeiou] matches any of the five vowels, [,.:;!?] matches some punctuation, and so on. The significance of the special regular expression characters—.|(){+^$*?—is turned off inside the brackets. However, normal string substitution still occurs, so (for example) \b represents a backspace character, and \n represents a new line.

```
show_regexp('Price $ 12.', /[aeiou]/)      # => Pr->i<-ce $ 12.
show_regexp('Price $ 12.', /[0123456789]/) # => Price $ ->1<-2.
show_regexp('Price $ 12.', /[$.]/)         # => Price ->$<- 12.
```

Sequence	Logical intent	Characters matched
\d	Decimal digit	(?a), (?d) → [0-9] (?u) → *Decimal_Number*
\D	Any character except a decimal digit	All characters not matched by \d
\h	Hexadecimal digit character	[0-9a-fA-F]
\H	Any character except a hex digit	All characters not matched by \h
\R	A generic linebreak sequence.	Matches any ASCII or Unicode linebreak. May also match the two characters \r\n.
\s	Whitespace	(?a), (?d) → [␣\t\r\n\f\v] (?u) → [\t\n\r\x{000B}\x{000C}\x{0085}] plus *Line_Separator, Paragraph_Separator, Space_Separator*
\S	Any character except whitespace	Any character not matched by \s
\w	A "word" character (really, a programming language identifier)	(?a), (?d) → [a-zA-Z0-9_] (?u) → *Letter, Mark, Number ,Connector_Punctuation*
\W	Any character except a word character	Any character not matched by \w
\X	An extended Unicode grapheme (two or more characters that combine to form a single visual character).	

Table 2—Character class abbreviations

For some of these classes, the meaning depends on the character set mode selected for the pattern. In these cases, the different options are shown like this:

(?a), (?d) → [a-zA-Z0-9_]
(?u) → *Letter, Mark, Number, Connector_Punctuation*

In this case, the first line applies to ASCII and default modes, and the second to unicode. In the second part of each line, the [...] is a conventional character class. Words in italic are Unicode character classes.

Within the brackets, a sequence such as c_1-c_2 represents all the characters from c_1 to c_2 in the current string encoding:

```
a = 'see [The PickAxe-page 123]'
show_regexp(a, /[A-F]/)      # => see [The Pick->A<-xe-page 123]
show_regexp(a, /[A-Fa-f]/)   # => s->e<-e [The PickAxe-page 123]
show_regexp(a, /[0-9]/)      # => see [The PickAxe-page ->1<-23]
show_regexp(a, /[0-9][0-9]/) # => see [The PickAxe-page ->12<-3]
```

It's common to see [a-zA-Z] to represent all the English letters or [0-9] to represent all the digits.

You can negate a character class by putting an up arrow (^, sometimes called a *caret*) immediately after the opening bracket: